

Full-Stack Form Application — Analysis & Dockerization Guide

Table of Contents

1. Architecture Overview
 2. Backend — Spring Boot
 3. Frontend — React (CRA)
 4. How the API Works
 5. Connecting Frontend to Backend
 6. Problems Without Docker
 7. Dockerization
 8. Changes Made & Issues Resolved
 9. Final Flow
-

1. Architecture Overview

Browser (localhost)

React Frontend (Nginx :80)	HTTP (Tomcat :8080)	Spring Boot Backend
Serves static HTML/JS/CSS		REST API POST /api/users

This is a simple form app: user submits name + age via a React form, the data is sent as JSON to a Spring Boot backend, processed, and a response string is displayed back on the page.

2. Backend — Spring Boot

2.1 Project Structure

form-app/		
pom.xml		← Maven build file (dependencies, plugins)
mvnw / mvnw.cmd		← Maven wrapper (build without installing Maven)
src/		
main/		
java/com/example/form_app/		
FormAppApplication.java		← Entry point
controller/UserController.java		← REST endpoint
service/UserService.java		← Business logic

dto/UserRequest.java	← Request data structure
resources/application.properties	← Config (port, app name)
test/.../FormAppApplicationTests.java	← Smoke test

2.2 pom.xml — What It Does

Defines the project and its dependencies:

Dependency	Purpose
spring-boot-starter-parent (v4.1.0)	Parent POM — provides default Spring Boot configs
spring-boot-starter-thymeleaf	Template engine (included but unused here)
spring-boot-starter-webmvc	Core web library — enables <code>@RestController</code> , <code>@PostMapping</code> , etc.
spring-boot-devtools	Auto-restart during development
spring-boot-starter-webmvc-test	Testing utilities

The `spring-boot-maven-plugin` packages everything into a single executable JAR.

Java version: 21

2.3 application.properties — Configuration

```
spring.application.name=form-app
server.port=8080
```

- `spring.application.name` — Sets the app name (used in logs, service discovery)
- `server.port=8080` — Tells Tomcat to listen on port 8080

2.4 FormAppApplication.java — Entry Point

```
@SpringBootApplication
public class FormAppApplication {
    public static void main(String[] args) {
        SpringApplication.run(FormAppApplication.class, args);
    }
}
```

- `@SpringBootApplication` is a shortcut for `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`
- It tells Spring: “scan this package and sub-packages for beans, auto-configure based on dependencies”
- `SpringApplication.run()` starts the embedded Tomcat server

2.5 UserRequest.java — DTO (Data Transfer Object)

```
public class UserRequest {
    private String name;
    private int age;
    // getters and setters
}
```

- A simple POJO that defines the shape of incoming JSON
- When Spring sees `@RequestBody UserRequest`, it automatically converts the JSON body `{"name": "John", "age": 25}` into this Java object
- Getters/setters are required for Jackson (JSON parser) to populate the fields

2.6 UserService.java — Business Logic

```
@Service
public class UserService {
    public String processUser(String name, int age) {
        return "Received " + name + " who is " + age + " years old";
    }
}
```

- `@Service` marks it as a Spring-managed bean
- Contains the only business logic: formats a response string
- In a real app, this is where you'd save to a database, call external APIs, etc.

2.7 UserController.java — The REST Endpoint

```
@RestController // Tells Spring: this class handles HTTP requests, responses
@RequestMapping("/api/users") // All routes in this class start with /api/users
@CrossOrigin(origins = "http://localhost") // Allows browser requests from http://localhost
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping // Handles POST requests to /api/users
    public String addUser(@RequestBody UserRequest request) {
        return userService.processUser(request.getName(), request.getAge());
    }
}
```

Key annotations explained:

Annotation	What it does
<code>@RestController</code>	Combines <code>@Controller</code> + <code>@ResponseBody</code> — every method return value goes directly in HTTP response body (no view resolution)
<code>@RequestMapping("/api/users")</code>	Sets the base path for all endpoints in this controller
<code>@CrossOrigin(origins = "http://localhost")</code>	Adds CORS headers to allow the browser to accept responses when the frontend is on a different origin
<code>@PostMapping</code>	Maps only POST requests to this method
<code>@RequestBody</code>	Tells Spring to deserialize the JSON request body into <code>UserRequest</code>

Constructor injection: Instead of `@Autowired`, the controller receives `UserService` through its constructor — Spring automatically provides it.

3. Frontend — React (CRA)

3.1 Project Structure

form/	
package.json	← Dependencies & scripts
public/index.html	← HTML template
src/	
index.js	← React entry point (renders App into DOM)
App.js	← Root component
UserForm.js	← Form with input fields + submit
ResponseMessage.js	← Displays server response

3.2 package.json — What It Does

```
{
  "dependencies": {
    "react": "^19.2.7",
    "react-dom": "^19.2.7",
    "react-scripts": "5.0.1"
  },
  "scripts": {
    "start": "react-scripts start",    ← Dev server on port 3000
    "build": "react-scripts build",    ← Production build → static/ folder
  }
}
```

- **Create React App (CRA)** — a toolchain that bundles React code using Webpack
- **react-scripts** handles all config (Babel, Webpack, dev server) — zero config needed
- **npm start** runs a development server with hot reload
- **npm run build** generates optimized static files in the **build/** directory

3.3 App.js — Root Component

```
function App() {
  const [message, setMessage] = useState("");
  return (
    <div>
      <UserForm setMessage={setMessage} />
      <ResponseMessage message={message} />
    </div>
  );
}
```

- `useState("")` creates a state variable `message` (initially empty string)
- `setMessage` is the function to update it
- `UserForm` receives `setMessage` — so it can update the message when it gets a response
- `ResponseMessage` receives `message` — so it can display it

Data flow: `UserForm` → (gets response) → calls `setMessage(result)` → `App` re-renders → `ResponseMessage` shows new message

3.4 UserForm.js — The Form & API Call

```
function UserForm({ setMessage }) {
  const [name, setName] = useState("");
  const [age, setAge] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault(); // Stop page reload

    const response = await fetch(
      "http://localhost:8080/api/users", // Backend URL
      {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name, age }),
      }
    );

    const result = await response.text(); // Read response as plain text
  }
}
```

```

        setMessage(result);                                // Pass it up to App
    };

    return (
        <form onSubmit={handleSubmit}>
            <input type="text" value={name} onChange={(e) => setName(e.target.value)} />
            <input type="number" value={age} onChange={(e) => setAge(e.target.value)} />
            <button type="submit">Submit</button>
        </form>
    );
}

```

Step-by-step flow: 1. User types name/age → `onChange` updates state (`setName`, `setAge`) → React re-renders with new values 2. User clicks Submit → `handleSubmit` runs 3. `e.preventDefault()` — prevents the browser from reloading the page 4. `fetch("http://localhost:8080/api/users", {...})` sends a POST request with JSON body 5. `response.text()` reads the server's response as plain text 6. `setMessage(result)` passes the response up to `App.js` → `ResponseMessage` displays it 7. Name/age fields are cleared (`setName("")`, `setAge("")`)

3.5 ResponseMessage.js — Display Component

```

function ResponseMessage({ message }) {
    return (
        <div>
            <h3>Server Response</h3>
            <p>{message}</p>
        </div>
    );
}

```

- A pure presentational component — receives `message` as a prop and renders it
- Initially empty; after form submission, shows something like “Received John who is 25 years old”

4. How the API Works

Request/Response Cycle

1. User fills form and clicks Submit
2. `fetch()` sends POST request to:
`http://localhost:8080/api/users`

```
Headers: Content-Type: application/json
Body:    { "name": "John", "age": 25 }
```

3. Spring Boot receives at Tomcat (port 8080)
4. DispatcherServlet routes to UserController.addUser()
5. Jackson deserializes JSON → UserRequest object
(name="John", age=25)
6. UserController calls userService.processUser(name, age)
7. UserService returns: "Received John who is 25 years old"
8. Spring writes this string to the HTTP response body
(Content-Type: text/plain)
9. Browser receives "Received John who is 25 years old"
10. React calls setMessage() → page shows the response

HTTP Details

- **Method:** POST
 - **URL:** http://localhost:8080/api/users
 - **Request Header:** Content-Type: application/json
 - **Request Body:** {"name": "John", "age": 25}
 - **Response:** "Received John who is 25 years old" (plain text)
-

5. Connecting Frontend to Backend

Without Docker — The Problems We Found

Problem 1: URL Path Mismatch - Frontend was calling http://localhost:8080/users

- Backend expected /api/users (because of @RequestMapping("/api/users"))

- Result: **404 Not Found**

Problem 2: CORS (Cross-Origin Resource Sharing) - Frontend runs on http://localhost:3000 (CRA dev server) - Backend runs on http://localhost:8080 - Browsers block requests from one origin to another unless the server explicitly allows it - Backend's @CrossOrigin originally allowed only http://localhost:5173 (Vite's port) - Result: **CORS error** — browser refused to read the response

What is CORS? - A browser security mechanism - When a web page at

`http://localhost:3000` tries to fetch data from `http://localhost:8080`, the browser sends a **preflight OPTIONS request** first - The server must respond with `Access-Control-Allow-Origin: http://localhost:3000` header - If missing or mismatched, the browser blocks the actual request

The Fixes Applied: 1. Changed fetch URL from `/users` to `/api/users`
2. Changed `@CrossOrigin(origins = "http://localhost:5173")` to `@CrossOrigin(origins = "http://localhost")` — but note: when using `npm start` (CRA dev server on port 3000), it should be `"http://localhost:3000"`. When using Docker with Nginx on port 80, it should be `"http://localhost"`

6. Problems Without Docker

Problem	Description
Environment differences	Must have Java 21 + Node.js 20 installed locally
Manual setup	Start backend (<code>./mvnw spring-boot:run</code>), start frontend (<code>npm start</code>) — 2 terminals
CORS issues	Different ports cause browser CORS errors
Port conflicts	If another app uses 8080 or 3000, things break
“Works on my machine”	Different OS/versions cause unpredictable issues

7. Dockerization

7.1 What is Docker?

Docker packages an application + all its dependencies into a **container image**. This image can run on any machine with Docker installed — the app behaves identically everywhere.

Key concepts: - **Image** — a read-only template (like a snapshot of the app + OS) - **Container** — a running instance of an image - **Dockerfile** — a recipe for building an image - **docker-compose.yml** — defines multiple containers that work together

7.2 Backend Dockerfile

```
FROM eclipse-temurin:21-jdk      # Base image with Java 21
WORKDIR /app                    # Working directory inside container
```



```

COPY . . . # Copy all source files into /app
RUN ./mvnw package -DskipTests # Build the JAR (skip tests for speed)
EXPOSE 8080 # Tell Docker this container listens on 8080
CMD ["java", "-jar", "target/form-app-0.0.1-SNAPSHOT.jar"] # Run the app

```

Step-by-step: 1. Start from an official Java 21 image (`eclipse-temurin:21-jdk`)
 2. Set the working directory to `/app` 3. Copy the entire project into the container 4. Run Maven to compile and package into a JAR (in `target/` folder)
 5. Expose port 8080 (documentation — doesn't actually publish the port) 6. When the container starts, run the JAR

Why `./mvnw`? The Maven Wrapper (`mvnw`) downloads the correct Maven version automatically — no need to install Maven globally.

7.3 Frontend Dockerfile

```

# ---- Build Stage ----
FROM node:20 AS build # Base image with Node.js 20
WORKDIR /app # Working directory
COPY package*.json ./ # Copy only package files first
RUN npm install # Install dependencies (cached unless package.json changes)
COPY . . # Copy the rest of the source code
RUN npm run build # Build static files → /app/build

# ---- Serve Stage ----
FROM nginx:alpine # Lightweight Nginx image
COPY --from=build /app/build /usr/share/nginx/html # Copy built files to Nginx
EXPOSE 80 # Nginx listens on port 80
CMD ["nginx", "-g", "daemon off;"] # Start Nginx in foreground

```

Multi-stage build explained: - **Stage 1 (build):** Uses Node.js to install dependencies and build the React app - **Stage 2 (serve):** Uses a tiny Nginx image — copies only the built static files - Result: The final image is small (~25MB instead of ~1.5GB with Node.js)

Why copy `package*.json` before the rest? Docker caches layers. If `package.json` hasn't changed, Docker reuses the cached `npm install` layer — much faster rebuilds.

7.4 docker-compose.yml

```

version: '3'
services:
  backend:
    build: ../SpringBoot/form/form-app # Build from this Dockerfile location
    ports:
      - "8080:8080" # Host:Container port mapping

```

```

frontend:
  build: ./8.React/form                # Build from this Dockerfile location
  ports:
    - "80:80"                          # Host:Container port mapping
  depends_on:
    - backend                          # Start backend first

```

Port mapping syntax: host-port:container-port - "8080:8080" → Traffic to your machine's port 8080 goes to container's port 8080 - "80:80" → Traffic to your machine's port 80 goes to container's port 80

depends_on: Docker starts the backend container before the frontend. But this does NOT wait for Spring Boot to be ready — just for the container to start.

8. Changes Made & Issues Resolved

Issue 1: Port Already in Use

Error:

Cannot start service backend: failed to bind host port 0.0.0.0:8080/tcp: address already in use

Cause: A Spring Boot instance was still running on the host machine, occupying port 8080.

Fix: Stopped the local Spring Boot process (Ctrl+C in its terminal, or `sudo lsof -i :8080 → kill <PID>`).

Issue 2: backend Hostname Not Resolvable

Error:

POST http://backend:8080/api/users net::ERR_NAME_NOT_RESOLVED

Cause: The fetch URL was `http://backend:8080/api/users` — `backend` is a Docker internal hostname that exists only inside the Docker network. The browser runs on the host machine and cannot resolve it.

Fix: Changed the fetch URL to `http://localhost:8080/api/users` — since Docker maps port 8080 from the container to the host, the browser can reach the backend via localhost.

Issue 3: CORS Error After Docker

Error:

Access to fetch at 'http://localhost:8080/api/users' from origin 'http://localhost' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

Cause: The frontend origin is now `http://localhost` (Nginx on port 80), but `@CrossOrigin` still allowed `http://localhost:5173`.

Fix: Changed `@CrossOrigin(origins = "http://localhost:5173")` to `@CrossOrigin(origins = "http://localhost")`.

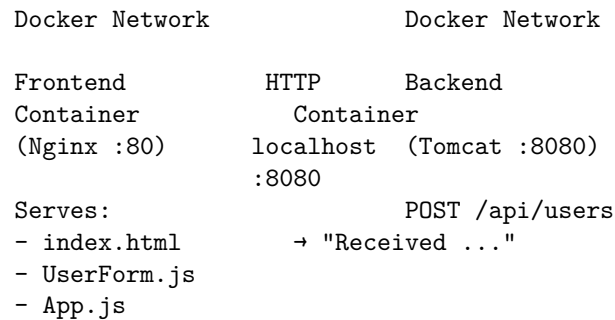
Summary of All Changes

File	Before	After	Why
UserForm.js:12	"http://localhost:8080/users"	"http://localhost:8080/api/users"	URI mismatch (missing /api)
UserController.java	<code>@CrossOrigin(origins = "http://localhost:5173")</code>	<code>@CrossOrigin(origins = "http://localhost")</code>	CORS origin changed from Vite port to Nginx port
New: docker-compose.yml	(didn't exist)	Created at Tutorial root	Define backend + frontend services
New: backend Dockerfile	(didn't exist)	Created in form-app/	Package Spring Boot in Docker
New: frontend Dockerfile	(didn't exist)	Created in form/	Build + serve React in Docker

9. Final Flow (Dockerized)

Your Machine

`http://localhost:80` (Open in browser)



How to run:

```

docker-compose up          # Start both containers
docker-compose down        # Stop both containers
docker-compose up --build  # Rebuild images and start

```

Commands Reference

Command	What it does
<code>docker-compose up</code>	Builds images (if needed) and starts containers
<code>docker-compose down</code>	Stops and removes containers
<code>docker-compose up --build</code>	Rebuilds images from source, then starts
<code>docker ps</code>	List running containers
<code>docker logs tutorial_backend_1</code>	View backend logs
<code>docker logs tutorial_frontend_1</code>	View frontend logs

Final File Locations

File	Path
Docker Compose	<code>/home/jo/Documents/Mashupstack/Tutorial/docker-compose.yml</code>
Backend Dockerfile	<code>/home/jo/Documents/Mashupstack/Tutorial/7.SpringBoot/form/Dockerfile</code>
Frontend Dockerfile	<code>/home/jo/Documents/Mashupstack/Tutorial/8.React/form/Dockerfile</code>
Backend Controller	<code>/home/jo/Documents/Mashupstack/Tutorial/7.SpringBoot/form/src/main/java/com/example/tutorial/BackendController.java</code>
Frontend Form	<code>/home/jo/Documents/Mashupstack/Tutorial/8.React/form/src/App.js</code>